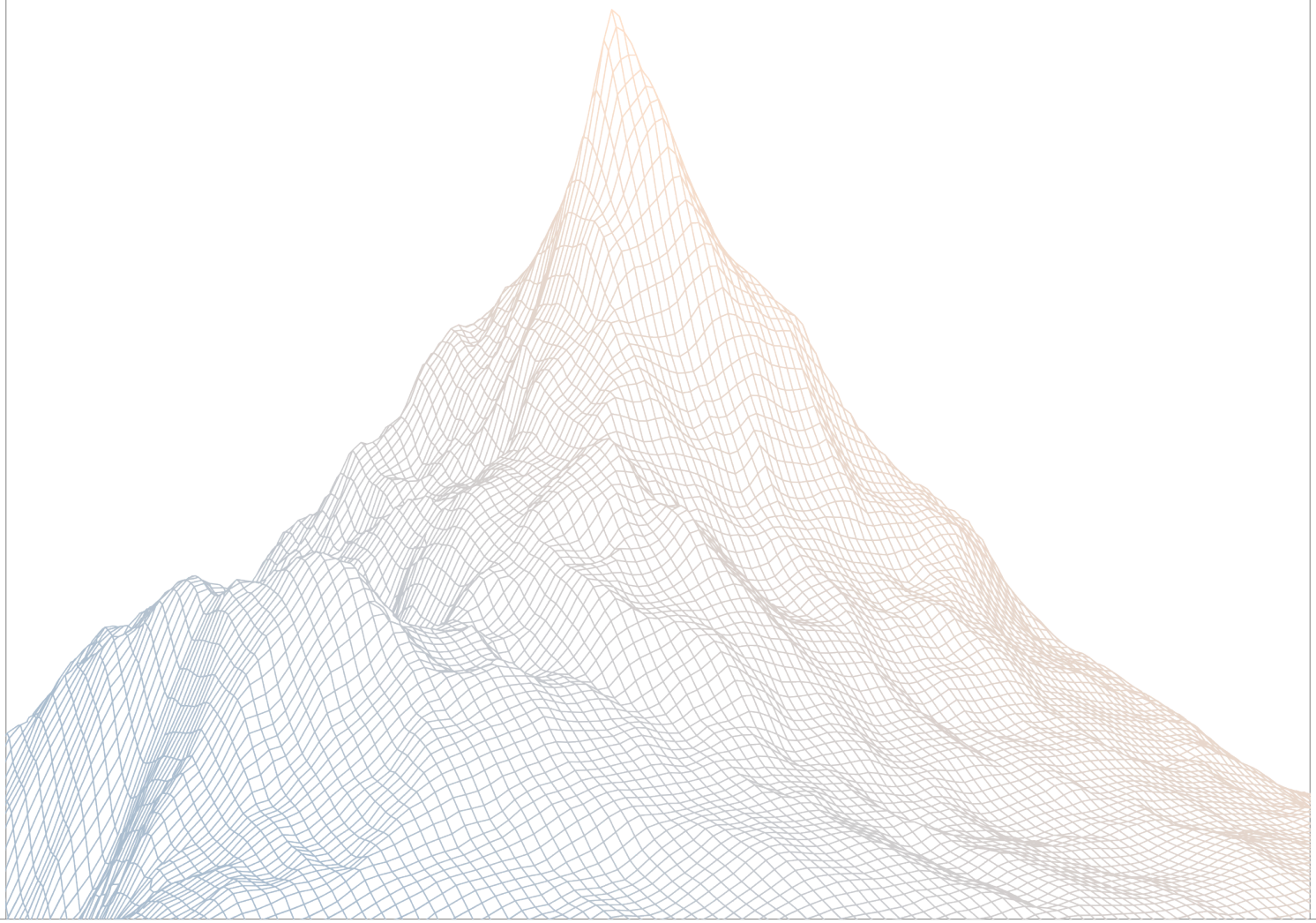
Zenith

Berachain

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: October 6th to October 7th, 2025

AUDITED BY: adriro
ether_sky

Contents

1 Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Berachain

Berachain is a high-performance EVM-Identical Layer 1 blockchain utilizing Proof-of-Liquidity (PoL) and built on top of the modular EVM-focused consensus client framework BeaconKit.

2.2 Scope

The engagement involved a review of the following targets:

Target	contracts-staking-pools
Repository	https://github.com/berachain/contracts-staking-pools
Commit Hash	`5da5d1b124424addc0764ae1138c508bb17c4972`
Files	`Changes in PR-73`

2.3 Audit Timeline

October 6, 2025	Audit start
October 7, 2025	Audit end
October 14, 2025	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	2
High Risk	0
Medium Risk	4
Low Risk	1
Informational	0
Total Issues	**7**

2.5 Security Note

Considering the number of issues identified, it is statistically likely that there are more complex bugs still present that could not be identified given the time-boxed nature of this engagement. It is recommended that a follow-up audit and development of a more complex stateful test suite be undertaken prior to continuing to deploy significant monetary capital to production.

3

Findings Summary

ID	Description	Status
C-1	Withdrawals can be permissionlessly finalized	Resolved
C-2	Rounding issues could block undelegations	Resolved
M-1	There is no validation of the request ID in the completeWithdrawal function	Resolved
M-2	The yield can also be claimed by the owner as part of the deposited amounts instead of being treated as actual yield	Resolved
M-3	The refunded amounts are not accounted for when depositing assets into the pool	Resolved
M-4	Overpaid fees are not refunded	Resolved
L-1	Incorrect yield calculation while delegated funds are being withdrawn	Resolved

4

Findings

4.1 Critical Risk

A total of 2 critical risk findings were identified.

[C-1] Withdrawals can be permissionlessly finalized

SEVERITY: Critical	IMPACT: High
STATUS: Resolved	LIKELIHOOD: High

Target

- DelegationHandler.sol

Description:

After a withdrawal request has been queued, the DelegationHandler contract expects it to be eventually finalized using the `completeWithdrawal()` function. This is necessary to properly account for the withdrawn amount or send the yield to the validator admin.

However, requests in the WithdrawalVault can be finalized by any account, not just the owner of the request. If a request that originated in the DelegationHandler is finalized by calling the WithdrawalVault directly, the logic in `completeWithdrawal()` will never be executed, resulting in accounting issues and loss of funds.

Recommendations:

In `completeWithdrawal()`, skip the call to `finalizeWithdrawalRequest()` if the withdrawal has already been exercised. Additionally, mark the request as completed locally to avoid replaying the action.

Berachain: Resolved with @705b1c723a....

Zenith: Verified.

[C-2] Rounding issues could block undelegations

SEVERITY: Critical	IMPACT: High
STATUS: Resolved	LIKELIHOOD: High

Target

- DelegationHandler.sol

Description:

When delegated funds are requested, the implementation calculates the amount of used funds and requests a withdrawal by rounding the amount to gwei units.

```
147:         uint256 delegatedAmountUsed = delegatedAmount
    - delegatedAmountAvailable;
148:         uint64 delegatedAmountUsedInGWei = uint64(delegatedAmountUsed
    / 1 gwei);
149:         _withdrawalVault.requestWithdrawal{ value: msg.value }(pubkey,
    delegatedAmountUsedInGWei, msg.value);
```

When the request is finalized, the requested amount is added to the `delegatedAmountAvailable` variable.

```
179:         _withdrawalVault.finalizeWithdrawalRequest(requestId);
180:
181:         if (isYieldRedeemRequest[requestId]) {
182:             SafeTransferLib.safeTransferETH(msg.sender,
    assetsRequested);
183:         } else {
184:             delegatedAmountAvailable += assetsRequested;
185:         }
```

The issue is that `undelegate()` has a strict equality check to process the action. If the amount of delegated funds is not a multiple of gweis, then the requested amount will never reach the equality condition needed to undelegate and withdraw the funds.

```
79:     function undelegate() external onlyRole(DEFAULT_ADMIN_ROLE) {
80:         if (delegatedAmount ≠ delegatedAmountAvailable) {
```

```
81:             revert FundsNotAvailable();
82:         }
```

Recommendations:

Ensure that delegated funds are always a multiple of gwei or relax the conditions required to undelegate the funds.

Berachain: Resolved with @25dd74da866....

Zenith: Verified.

4.2 Medium Risk

A total of 4 medium risk findings were identified.

[M-1] There is no validation of the request ID in the completeWithdrawal function

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- DelegationHandler.sol

Description:

Anyone can finalize any `withdrawal request` using the `finalizeWithdrawalRequest` function.

- WithdrawalVault.sol#L131

```
function finalizeWithdrawalRequest(uint256 requestId)
    external nonReentrant whenNotPaused {
    _finalizeWithdrawalRequest(requestId);
}
```

As a result, in the `completeWithdrawal` function, the `requestId` could belong to another user, not this contract.

- DelegationHandler.sol#L184

```
function completeWithdrawal(uint256 requestId)
    external whenNotPaused auth(requestId) {
    IWithdrawalVault.WithdrawalRequest memory request
    = _withdrawalVault.getWithdrawalRequest(requestId);
    uint256 assetsRequested = request.assetsRequested;

    _withdrawalVault.finalizeWithdrawalRequest(requestId);
```

```
    if (isYieldRedeemRequest[requestId]) {
        SafeTransferLib.safeTransferETH(msg.sender, assetsRequested);
    } else {
        delegatedAmountAvailable += assetsRequested;
    }

    emit WithdrawalCompleted(requestId, assetsRequested, msg.sender);
}
```

In this function, the `delegatedAmountAvailable` still increases, even though no actual tokens were `withdrawn`.

Recommendations:

```
function completeWithdrawal(uint256 requestId)
    external whenNotPaused auth(requestId) {
    IWithdrawalVault.WithdrawalRequest memory request
    = _withdrawalVault.getWithdrawalRequest(requestId);
    uint256 assetsRequested = request.assetsRequested;

    uint256 beforeBalance = address(this).balance;

    _withdrawalVault.finalizeWithdrawalRequest(requestId);

    uint256 withdrawed = address(this).balance - beforeBalance;
    if (withdrawed != assetsRequested) revert();

    if (isYieldRedeemRequest[requestId]) {
        SafeTransferLib.safeTransferETH(msg.sender, assetsRequested);
    } else {
        delegatedAmountAvailable += assetsRequested;
    }

    emit WithdrawalCompleted(requestId, assetsRequested, msg.sender);
}
```

Berachain: Resolved with @705b1c723a....

Zenith: Verified.

[M-2] The yield can also be claimed by the owner as part of the deposited amounts instead of being treated as actual yield

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- DelegationHandler.sol

Description:

Suppose the `delegatedAmount` is `100`, `delegatedAmountAvailable` is `50`, and the `yield` is `60`. This means the total available `balance` is `110 (50 + 60)`.

When the `admin` calls the `requestDelegatedFundsWithdrawal` function to `withdraw` the `deposited amount` of `50`, there's a time delay before the `withdrawal` is completed, so `delegatedAmountAvailable` remains `50`.

- DelegationHandler.sol#L147

```
function requestDelegatedFundsWithdrawal()
    external payable onlyRole(DEFAULT_ADMIN_ROLE) {
    if (msg.value < 1) {
        revert InvalidAmount();
    }

    uint256 delegatedAmountUsed = delegatedAmount
    - delegatedAmountAvailable;
    uint64 delegatedAmountUsedInGWei = uint64(delegatedAmountUsed / 1 gwei);
    _withdrawalVault.requestWithdrawal{ value: msg.value }(pubkey,
    delegatedAmountUsedInGWei, msg.value);

    emit DelegatedFundsWithdrawalRequested(delegatedAmountUsed);
}
```

During this time, the available `balance` is `60`, allowing the `admin` to call `requestDelegatedFundsWithdrawal` again and `withdraw` another `50 assets`.

Recommendations:

Another `Withdrawal` could be prevented while the previous `withdrawal` is not completed.

Berachain: Resolved with @280d7761c18....

Zenith: Verified.

[M-3] The refunded amounts are not accounted for when depositing assets into the pool

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- DelegationHandler.sol

Description:

When depositing `assets` into the `pool`, some `assets` may be refunded to the caller.

- StakingPool.sol#L268

```
function _submit(address receiver)
    internal whenNotPaused returns (uint256 shares) {
   (uint256 updatedBufferedAssets, uint256 userDepositAmount,
    uint256 refundAmount, uint256 rewardsToCollect) =
        _computeDepositAmounts(msg.value);

    if (userDepositAmount > 0) {
        shares = previewDeposit(userDepositAmount);
        _mintShares(receiver, shares);
    }

    if (rewardsToCollect > 0) {
        _collectRewards(rewardsToCollect);
    }

    bufferedAssets = updatedBufferedAssets;

    _processDeposit();

    if (refundAmount > 0) {
@->     SafeTransferLib.safeTransferETH(msg.sender, refundAmount);
    }
}
```

However, in the `depositDelegatedFunds` function, these refunded amounts are not

accounted for, which can affect the `yield calculation`.

- DelegationHandler.sol#L136

```
function depositDelegatedFunds(uint256 amount)
    external onlyRole(VALIDATOR_ADMIN_ROLE) {
    if (amount == 0 || amount > delegatedAmountAvailable) {
        revert InvalidAmount();
    }

    IStakingPool(stakingPool).submit{ value: amount }(address(this));
    delegatedAmountAvailable -= amount;

    emit DelegatedFundsDeposited(amount);
}
```

Recommendations:

```
function depositDelegatedFunds(uint256 amount)
    external onlyRole(VALIDATOR_ADMIN_ROLE) {
    if (amount == 0 || amount > delegatedAmountAvailable) {
        revert InvalidAmount();
    }

    uint256 beforeBalance = address(this).balance;

    IStakingPool(stakingPool).submit{ value: amount }(address(this));

    uint256 deposited = beforeBalance - address(this).balance;

    delegatedAmountAvailable -= amount;
    delegatedAmountAvailable -= deposited;

    emit DelegatedFundsDeposited(amount);
    emit DelegatedFundsDeposited(deposited);
}
```

Berachain: Resolved with @3f978dc9d....

Zenith: Verified.

[M-4] Overpaid fees are not refunded

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- DelegationHandler.sol

Description:

In the `requestWithdrawal` and `requestRedeem` functions, any `overpaid fees` are refunded to the `msg.sender`.

- WithdrawalVault.sol#L233

```
function _withdraw(
    bytes memory pubkey,
    uint64 assetsInGWei,
    uint256 maxFeeToPay
)
    internal
    returns (uint256 requestId)
{
    if (!_isFullyExited[pubkey] && !isShortCircuit) {
        uint256 feeToPay = _getWithdrawalRequestFee();
        if (feeToPay > maxFeeToPay) {
            revert OverpaidFee();
        }

        if (msg.value < feeToPay) {
            revert InsufficientFee();
        }

        _triggerExecutionLayerWithdrawal(pubkey, assetsInGWei, feeToPay);
@->     _refundOverpaidFee(feeToPay, msg.sender);
    } else {
        // If the pool has fully exited or short circuit is triggered, refund
    the fee if any
        if (msg.value > 0) {
@->         SafeTransferLib.safeTransferETH(msg.sender, msg.value);
```

```
        }
    }
}
```

However, in the `DelegationHandler`, when the `admin` calls `requestDelegatedFundsWithdrawal` with some `fees`, the `overpaid portion` is refunded to the `DelegationHandler` instead of the original `caller`.

- DelegationHandler.sol#L149

```
function requestDelegatedFundsWithdrawal()
    external payable onlyRole(DEFAULT_ADMIN_ROLE) {
    if (msg.value < 1) {
        revert InvalidAmount();
    }

    uint256 delegatedAmountUsed = delegatedAmount
    - delegatedAmountAvailable;
    uint64 delegatedAmountUsedInGWei = uint64(delegatedAmountUsed / 1 gwei);
    _withdrawalVault.requestWithdrawal{ value: msg.value }(pubkey,
    delegatedAmountUsedInGWei, msg.value);

    emit DelegatedFundsWithdrawalRequested(delegatedAmountUsed);
}
```

The same issue occurs in the `requestYieldWithdrawal` function.

Recommendations:

```
function requestDelegatedFundsWithdrawal()
    external payable onlyRole(DEFAULT_ADMIN_ROLE) {
    if (msg.value < 1) {
        revert InvalidAmount();
    }

    uint256 delegatedAmountUsed = delegatedAmount
    - delegatedAmountAvailable;
    uint64 delegatedAmountUsedInGWei = uint64(delegatedAmountUsed / 1 gwei);

    uint256 beforeBalance = address(this).balance - msg.value;

    _withdrawalVault.requestWithdrawal{ value: msg.value }(pubkey,
    delegatedAmountUsedInGWei, msg.value);
```

```
        uint256 refunded = address(this).balance - beforeBalance;
        if (refunded) {
            SafeTransferLib.safeTransferETH(msg.sender, refunded);
        }

        emit DelegatedFundsWithdrawalRequested(delegatedAmountUsed);
    }
```

The same fix should be applied to the `requestYieldWithdrawal` function.

Berachain: Resolved with @9330d7ba....

Zenith: Verified.

4.3 Low Risk

A total of 1 low risk findings were identified.

[L-1] Incorrect yield calculation while delegated funds are being withdrawn

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- DelegationHandler.sol

Description:

The implementation of `requestYieldWithdrawal()` relies on the stBERA balance to calculate the amount of accumulated yield.

```
160:         uint256 sharesBalance
    = IStBera(stakingPool).balanceOf(address(this));
161:         uint256 delegatedAmountUsed = delegatedAmount
    - delegatedAmountAvailable;
162:         uint256 lockedShares
    = IStBera(stakingPool).previewWithdraw(delegatedAmountUsed);
163:
164:         uint256 redeemableShares = sharesBalance - lockedShares;
```

The `delegatedAmountUsed`, calculated as the difference of `delegatedAmount - delegatedAmountAvailable`, is used to define the portion of assets that are excluded from the total share balance to calculate the redeemable shares corresponding to the amount of yield.

Note that, while there is a pending withdrawal corresponding to delegated funds, the associated shares have already been burned, but the `delegatedAmountAvailable` variable has not been adjusted yet. This will lead to an inconsistency between `sharesBalance` and `lockedShares`, likely causing an overflow in line 164.

Recommendations:

Consider adjusting the `delegatedAmountUsed` variable to account for any pending withdrawal of delegated funds.

Berachain: Resolved with @280d7761....

Zenith: Verified.